STC89C52 Educational Microcontroller Simulator

Week 4 - Standalone vs Multi-Process Benchmark Results and Performance Analysis

Metric	Standalone	Multi-Process
Simulator processes	1	3
Execution measurement	19.758 ms*	171.519 ms
Memory usage	49.17 MB	139.54 MB
Observed peak CPU	~1.52%	~1.45%
IPC between components	None	TCP sockets

* Standalone execution value is the sum of the four individually measured CPU step intervals. The multi-process value is the Week 4 program-time measurement and includes the UI-driven multi-process execution path.

1. Benchmark Objective

The purpose of this benchmark was to compare the original standalone STC89C52 simulator with the Week 4 multi-process version. The comparison focused on execution time, memory usage, observed CPU usage, number of simulator processes, and the additional overhead introduced by process separation and inter-process communication.

The benchmark was intended to evaluate the performance trade-off introduced when the simulator was divided into independent UI, Core and Logging processes.

2. Test Setup and Methodology

The same four-instruction assembly program was used for both versions so that the functional workload remained consistent. The standalone simulator was executed as a single Java simulator process. The Week 4 version was executed as three independent simulator processes communicating through TCP sockets.

Property	Standalone	Multi-Process
Architecture	Single-process simulator	UI + Core + Logging processes
Process count	1 simulator process	3 simulator processes
IPC	Not required between components	TCP socket-based IPC
Logging architecture	Within standalone application	Independent Logging Process
Test workload	Same 4 instructions	Same 4 instructions

The Java processes belonging to VS Code and other development tools were excluded from the simulator memory comparison.

3. Benchmark Test Program

```
MOV A,#10
ADD A,#5
INC A
HALT
```

The expected final accumulator value was 16: MOV sets A to 10, ADD changes A to 15, INC changes A to 16, and HALT terminates execution.

4. Standalone Simulator Results

For the standalone simulator, execution timing was added around the CPU step operation because the existing UI performs continuous execution through repeated STEP operations. The measured instruction intervals were:

Instruction	Measured time
MOV A,#10	6.520 ms
ADD A,#5	9.949 ms
INC A	2.831 ms

Instruction	Measured time
HALT	0.458 ms
Total	**19.758 ms**

Standalone memory usage: 49.17 MB

Observed peak CPU usage during the short test: approximately 1.52%

5. Multi-Process Simulator Results

The Week 4 simulator ran as three independent simulator processes. The UI communicated with the Core using TCP socket IPC, while execution and system information was sent to the independent Logging Process.

Measurement	Result
Program time	171.519 ms
Last instruction time	15.887 ms
Combined memory of 3 simulator JVMs	139.54 MB
Observed peak CPU during short test	~1.45%
Number of simulator processes	3

6. Standalone vs Multi-Process Comparison

Metric	Standalone	Multi-Process	Observation
Execution measurement	19.758 ms*	171.519 ms	Multi-process path took longer
Memory usage	49.17 MB	139.54 MB	Multi-process used more memory
Observed peak CPU	~1.52%	~1.45%	Very similar in this short sample
Processes	1	3	Week 4 separates responsibilities
IPC	None	TCP sockets	Communication overhead is introduced

* The standalone value represents the sum of individually timed CPU step intervals, while the Week 4 value represents the UI-driven multi-process program path. The values therefore describe practical observed performance, but they are not perfectly identical timing scopes.

7. Execution-Time Analysis

The standalone simulator completed the measured CPU step intervals in a total of 19.758 ms, whereas the Week 4 multi-process program-time measurement was 171.519 ms.

Observed difference = 171.519 ms - 19.758 ms = 151.761 ms.

This 151.761 ms value should be described as observed multi-process execution overhead, not as pure TCP IPC overhead. The additional time can include socket communication, process scheduling, request/response handling, logging, UI-driven repeated STEP requests, state updates, and other multi-process coordination costs.

For this small workload, the standalone version is expected to be faster because execution remains within one simulator process and does not require coordination among independent processes.

8. Memory-Usage Analysis

The standalone Java simulator backend used 49.17 MB of memory. The three Week 4 simulator JVM processes together used 139.54 MB.

Memory increase = 139.54 MB - 49.17 MB = 90.37 MB.

The increase is consistent with the multi-process design because each independent Java process requires its own JVM runtime, process resources, communication objects, queues and supporting data structures. The additional memory is therefore a trade-off for process separation and modularity.

9. CPU-Usage Analysis

The observed peak CPU usage was approximately 1.52% for the standalone simulator and 1.45% for the multi-process simulator during the short sampling period.

The difference is very small and should not be interpreted as evidence that the multi-process design is more CPU-efficient. The benchmark program contains only four instructions and CPU usage was sampled at one-second intervals, so much of the actual execution completed between samples. These values are best reported as observed peaks from the test rather than as precise average CPU utilization.

10. Performance Trade-offs

Standalone Simulator	Multi-Process Simulator
Lower measured execution time for this small workload	Higher coordination and communication overhead
Lower memory usage	Higher memory usage because multiple JVMs are active
Simpler execution path	Clear separation of UI, Core and Logging responsibilities
No IPC required between simulator components	TCP socket communication between independent processes
Less process isolation	Improved modularity and process isolation

11. Measurement Limitations

The benchmark uses a very short four-instruction program, so startup, communication and scheduling costs form a relatively large portion of the measured time. The standalone timing was collected by summing four CPU step intervals, while the multi-process measurement represents the broader UI-driven execution path. For this reason, the execution-time values are useful for practical comparison of the tested implementations but should not be treated as a controlled microbenchmark of TCP alone.

CPU measurements are also limited by the one-second sampling interval. A longer-running benchmark with repeated trials would provide a more stable CPU average and allow calculation of mean, minimum and maximum execution times.

12. Conclusion

The benchmark shows a clear performance trade-off between the two simulator architectures. For the tested four-instruction workload, the standalone simulator required less measured execution time and less memory because all simulator operations occurred within a single process.

The Week 4 multi-process simulator required additional execution time and memory because the UI, Core and Logging responsibilities were separated into independent processes and coordinated

through TCP socket communication. However, this design provides important software-engineering benefits including modularity, process isolation, clearer separation of responsibilities and a structured IPC architecture.

Therefore, the Week 4 implementation demonstrates the intended trade-off: the standalone version is more lightweight for a small workload, while the multi-process version introduces measurable overhead in exchange for a more modular and independently coordinated system architecture.